

CARD 1

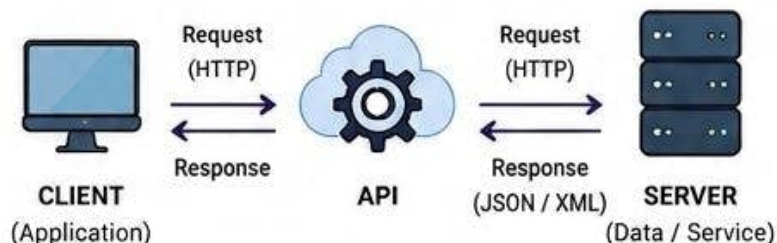
API – DEFINITION

★ The Bridge Between Applications ★

DEFINITION

API (Application Programming Interface) is a **set of rules and protocols** that allows different software applications to **communicate** with each other.

HOW API WORKS



COMMON USE CASES

- Search Data
- Retrieve Documents
- Trigger Workflows
- Book Tickets
- Process Payments
- Send Emails
- And many more...

★ REAL-WORLD EXAMPLE – SWIGGY

Swiggy doesn't cook food or deliver itself. It connects everyone using APIs.



INTERVIEW ANSWER (ONE-LINER)

API is a **communication interface** between two applications. It allows clients to request services from a server using **HTTP requests** and receive structured responses like **JSON or XML**.

QUICK REVISION

- ✓ API = Application Programming Interface
- ✓ Acts as a bridge between client & server
- ✓ Uses HTTP for communication
- ✓ Sends request → Gets response
- ✓ Responses are usually in JSON / XML
- ✓ Makes applications modular & scalable

KEY POINTS

- Defines how software components interact.
- Enables integration and automation.
- Improves efficiency and reusability.
- Foundation of web & mobile apps, AI tools, and modern systems.

INTERVIEW QUESTIONS

- 1 What is an API?
- 2 Why do we use APIs?
- 3 How does an API work?
- 4 What are some real-world examples of APIs?



MEMORY TIP: Think of API as a **WAITER** in a restaurant – you (client) place an order (request), the waiter (API) takes it to the kitchen (server) and brings back the food (response)!



HTTP METHODS & JWT

HTTP METHODS

GET	Retrieve data	GET /users
POST	Create new data	POST /users
PUT	Update entire resource	PUT /users/10
PATCH	Update specific fields	PATCH /users/10
DELETE	Delete data	DELETE /users/10

JWT (JSON WEB TOKEN)

HEADER

•

PAYLOAD

•

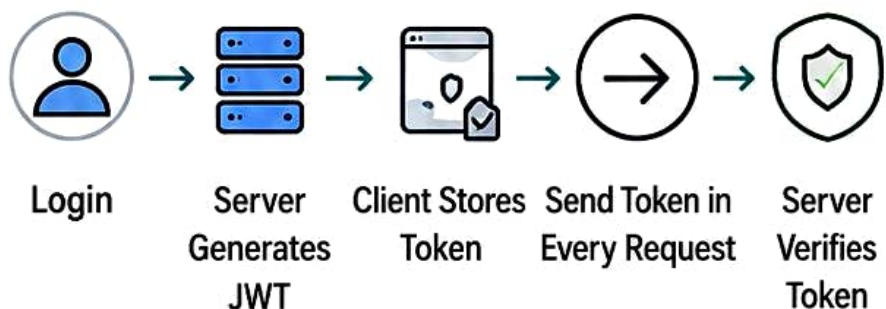
SIGNATURE

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

WHY JWT?

- ✓ Stateless
- ✓ Secure
- ✓ Compact
- ✓ Self-contained

JWT WORKFLOW



API ARCHITECTURE PATTERNS

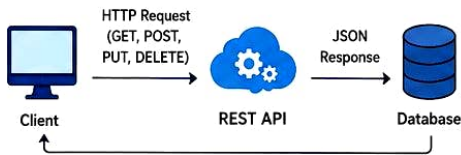
REST API (Representational State Transfer)

DEFINITION



REST API is an architectural style that enables communication between a client and a server using HTTP methods. It exchanges data mainly in JSON format and is lightweight, scalable, and easy to use.

VISUAL REPRESENTATION



DISADVANTAGES



- Multiple requests may be required to fetch related data.
- Over-fetching or under-fetching of data can occur.
- Versioning (v1, v2) increases maintenance.

USAGE



- Web applications
- Mobile applications
- E-commerce platforms
- Social media applications
- Public APIs

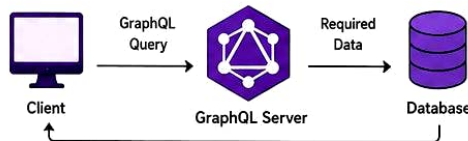
GRAPHQL API

DEFINITION



GraphQL is a query language for APIs that allows clients to request exactly the data they need in a single request, improving efficiency and reducing network traffic.

VISUAL REPRESENTATION



DISADVANTAGES



- More complex to implement than REST.
- Queries can become expensive without optimization.
- Caching is more difficult than REST APIs.

USAGE



- Single Page Applications (SPA)
- Mobile apps with limited bandwidth
- Social media platforms
- Real-time dashboards
- Applications requiring flexible data fetching

SOAP API (Simple Object Access Protocol)

DEFINITION



SOAP is a protocol for exchanging structured information using XML messages. It provides built-in security, reliability, and transaction support, making it suitable for enterprise applications.

VISUAL REPRESENTATION



DISADVANTAGES



- XML messages are large and slower to process.
- More complex than REST.
- Higher bandwidth usage.

USAGE



- Banking systems
- Healthcare applications
- Government services
- Enterprise software
- Secure financial transactions

API ARCHITECTURE PATTERNS

gRPC API (Google Remote Procedure Call)

DEFINITION



gRPC is a high-performance, open-source RPC framework developed by Google. It uses Protocol Buffers (binary format) over HTTP/2 for fast and efficient communication between services.

VISUAL REPRESENTATION



DISADVANTAGES



- Requires HTTP/2, which is not supported by all environments.
- Not easily testable in browsers (needs gRPC-Web).
- Complex setup and learning curve (Protocol Buffers).

USAGE



- Microservices communication
- High-performance applications
- Real-time streaming services
- IoT applications
- Internal system communication

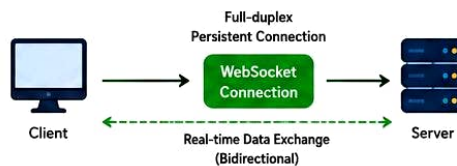
WEBSOCKETS

DEFINITION



WebSockets provide full-duplex, persistent communication channels over a single TCP connection. They allow real-time data exchange between client and server.

VISUAL REPRESENTATION



DISADVANTAGES



- More complex to implement than HTTP.
- Not suitable for request/response style APIs.
- Stateful connections may consume more server resources.

USAGE



- Live chat applications
- Real-time dashboards
- Online gaming
- Streaming applications
- Collaborative tools (e.g., whiteboards)

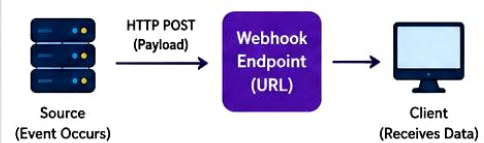
WEBHOOKS

DEFINITION



Webhooks are user-defined HTTP callbacks triggered by events. Instead of polling, the server sends real-time data to the client when something happens.

VISUAL REPRESENTATION



DISADVANTAGES



- Depends on endpoint availability (if endpoint is down, data may be lost).
- Security risks if endpoint is not verified.
- Harder to debug and test.

USAGE

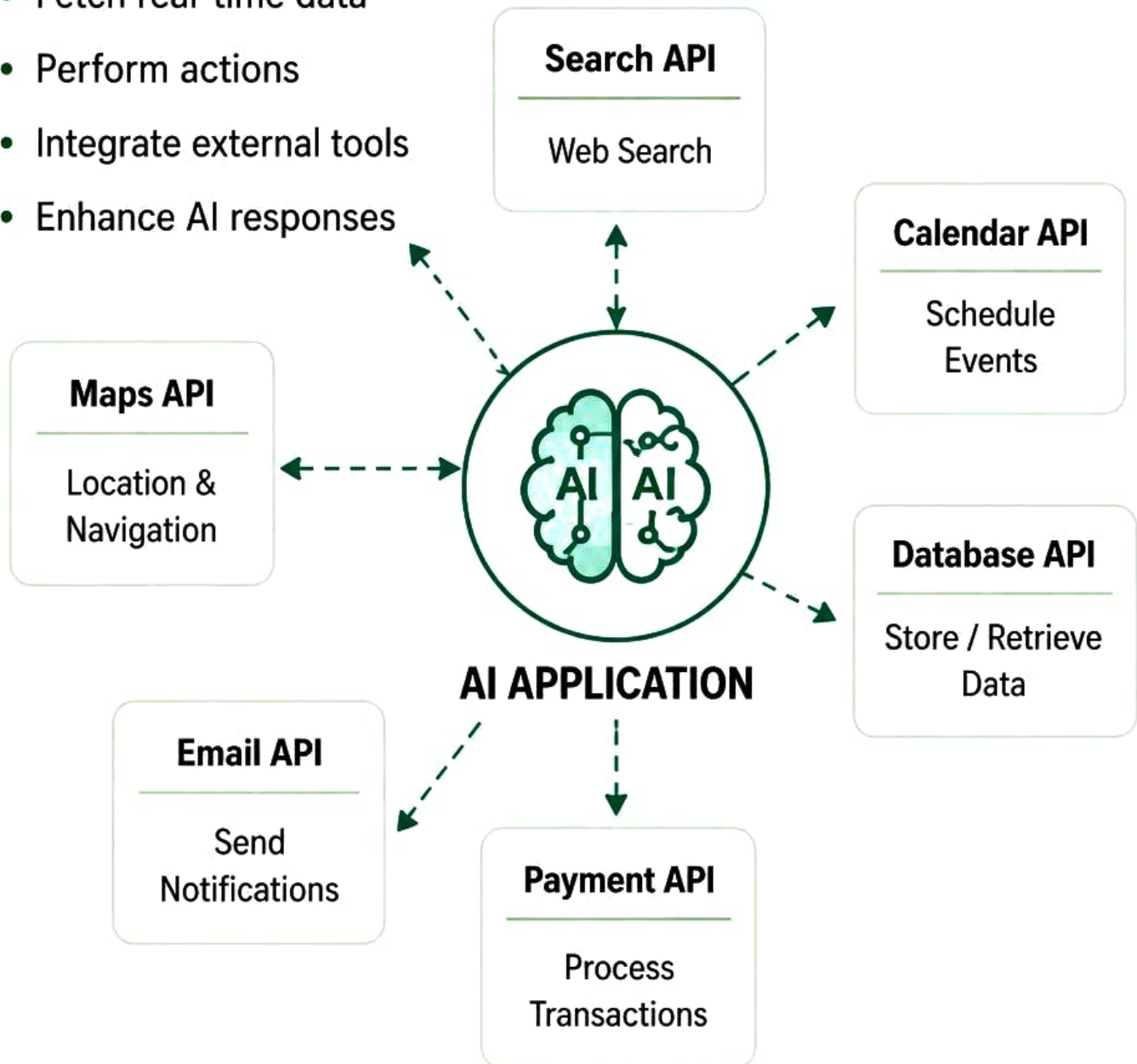


- Payment gateways (e.g., Stripe)
- CI/CD pipelines (e.g., GitHub)
- Real-time notifications
- Third-party integrations
- Event-driven architectures

API INTEGRATED IN AI APPLICATIONS

HOW AI APPS USE APIS

- Fetch real-time data
- Perform actions
- Integrate external tools
- Enhance AI responses



EXAMPLE: AI Travel Assistant → Uses Maps API, Booking API, Weather API, Payment API to plan & book a trip.

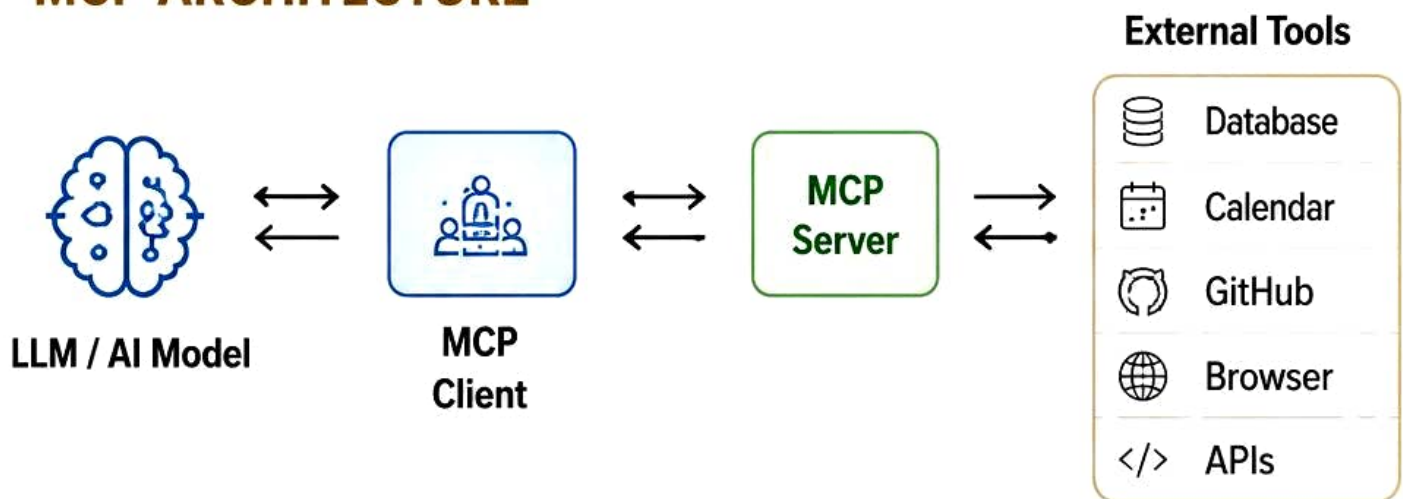
CARD 6

MCP (MODEL CONTEXT PROTOCOL) & MCP WORKFLOW

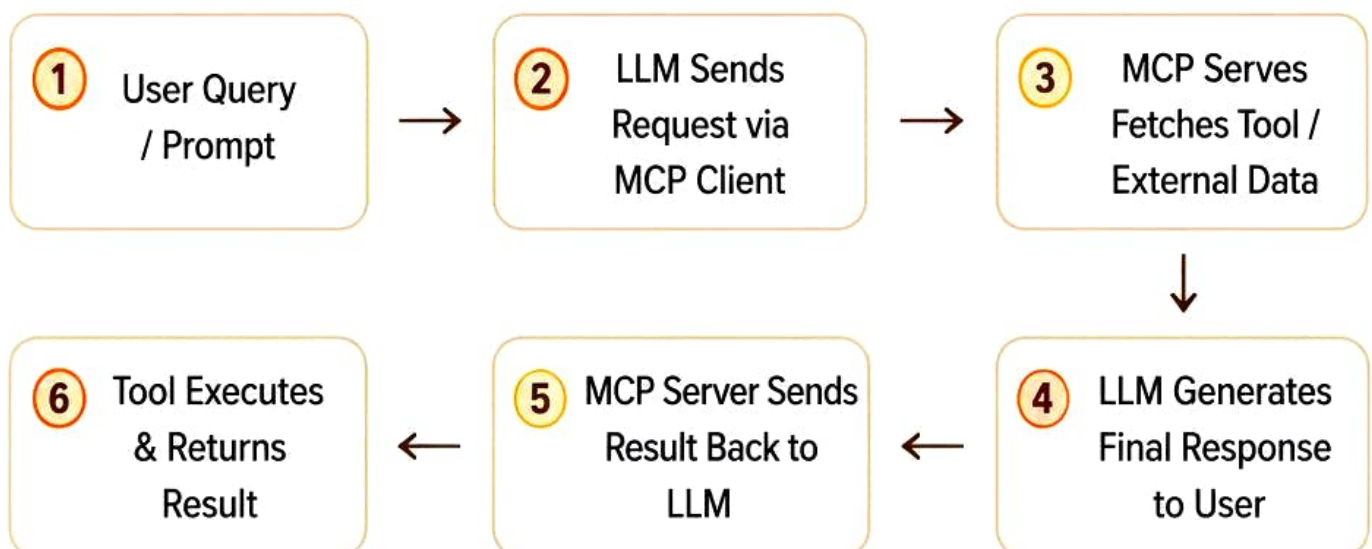
WHAT IS MCP?

MCP is a protocol that allows AI models (like LLMs) to connect securely with external tools, systems and data sources in a standardized way.

MCP ARCHITECTURE



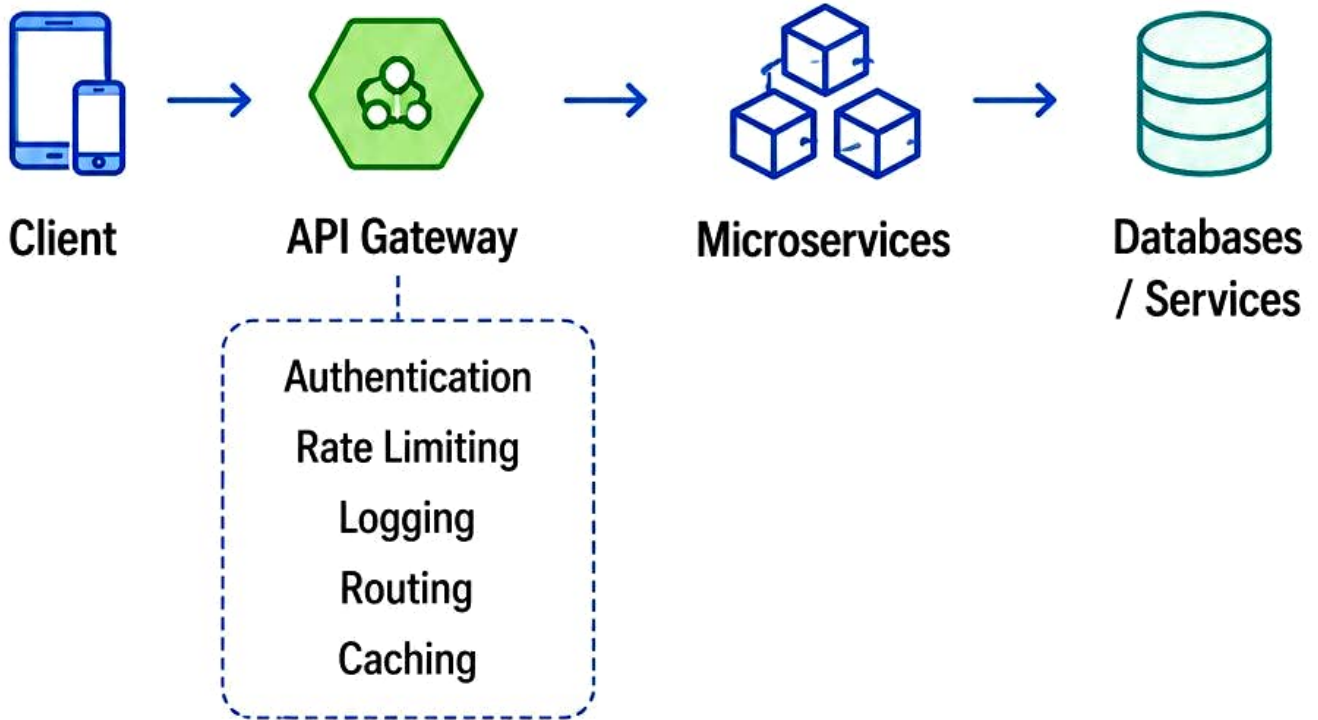
MCP WORKFLOW



Build Once • Reuse Everywhere • Standardized Integrations

API INFRASTRUCTURE & CORE WORKFLOW

API INFRASTRUCTURE



CORE WORKFLOW



Secure • Scalable • Reliable • Maintainable

API FRAMEWORKS

FRAMEWORK	LANGUAGE	BEST FOR	KEY FEATURES
 Flask	Python	Small APIs / Microservices	• Lightweight • Easy to learn • Flexible
 FastAPI	Python	Modern, Fast APIs	• Very Fast • Async Support • Autodoc (Swagger)
 Django REST Framework	Python	Large Django Projects	• Secure • Feature Rich • Scalable
 Express.js	Node.js	JavaScript APIs	• Minimal • Fast • Middleware
 Spring Boot	Java	Enterprise Applications	• Robust • Scalable • Production Ready
 ASP.NET Core	C#	Microsoft Ecosystem	• Cross Platform • High Performance • Secure

“Choose the right framework based on your project needs, team expertise and scalability.”